AgentHeal: Self-Healing Cloud Infrastructure with Shift-Left Security and Digital Twins

Aadi Haldar, Raghuram Sekar, Aaditya Paul, Shravan Rajesh Menon
Department of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham, Coimbatore, India

***Abstract*—Kubernetes-based microservice deployments today suffer from two closely linked problems that existing tools handle in isolation. Code vulnerabilities slip through during development because static analyzers have no awareness of what is actually happening in production. And when something breaks at runtime, autoscalers react to surface-level CPU numbers without understanding the real cause, which often means they make things worse. We built AgentHeal to tackle both problems together through a single closed-loop system. Before code is merged, a GitHub App scans pull requests for OWASP Top 10:2021 violations using AST-level analysis, checks dependencies for known CVEs, scores each PR for compliance, and uses a Gemini 2.0 Flash reasoning agent to automatically generate a fix branch when issues are found. After deployment, a live digital twin tracks cluster telemetry, flags anomalies using an Isolation Forest model in 2.26 ms, explains which metric is driving the problem via KernelSHAP, and runs two agents in parallel: a SimiFed reinforcement learning agent and an LLM agent. Before anything is executed on the cluster, the proposed action is validated through a SimPy $M/M/c$ discrete-event simulation to ensure it will not trigger a cascade failure. We tested AgentHeal against the Google Online Boutique benchmark across four fault injection scenarios with $N = 15$ trials each. Mean detection latency was 2.26 ms ($\sigma = 0.37$ ms) with a 98.3% detection rate, and end-to-end recovery averaged 2.98 s ($\sigma = 0.23$ s), a greater than 98% improvement over threshold-based autoscalers. The static analysis pipeline achieved 98.5% precision and 97.5% recall over 50 synthetic pull requests, and fully remediated 14 injected vulnerabilities across all OWASP categories in a live GitHub pull request test.**

***Index Terms*—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, OWASP Top 10, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing.**

I. Introduction

Teams running microservice applications on Kubernetes face two problems that compound each other in practice. The first is that code changes introduce security flaws and misconfigurations that slip into production undetected; the 2024 State of DevOps report puts over 60% of unplanned outages down to preventable defects introduced during development [1]. The second is that when something does break in production, the standard response is either a human on-call engineer who may take 30 to 60 minutes to act, or the Kubernetes HPA which blindly scales replicas based on aggregate CPU without any understanding of what is actually wrong. Neither is adequate when the failure is a deadlock, a memory leak, or a cascading dependency collapse [1], [9].

The two disciplines that are supposed to handle these problems, Shift-Left static analysis and Shift-Right runtime operations, have historically had nothing to do with each other. Tools like Bandit [6] and SonarQube [7] scan code during CI/CD but have no idea what the cluster looks like at runtime and cannot write fixes. Runtime observability and chaos engineering [24], [23] tell you when something is broken but offer no pre-merge prevention and no root-cause attribution beyond metric thresholds. Recent work on federated digital twin management [12] and LLM-driven program repair [15] has pushed each side forward, but nothing has connected them end-to-end with a safety verification step before acting.

We built AgentHeal specifically to close this gap. On the pre-merge side, a GitHub App intercepts every pull request, scans code structure against all ten OWASP Top 10:2021 categories, checks dependency files for known CVEs, and scores the result numerically. When a PR fails, a Gemini 2.0 Flash agent generates a corrective patch and opens a linked fix branch. The CI/CD quality gate blocks the merge until the score recovers. On the runtime side, a digital twin polls Prometheus telemetry every five seconds, runs the snapshot through an in-memory Isolation Forest (mean detection time 2.26 ms), identifies which metric is responsible via KernelSHAP, and sends the attribution to two agents running in parallel: a tabular SimiFed Q-learning agent and a Gemini LLM ReAct agent. Before anything touches the live cluster, the proposed action is simulated in SimPy using an $M/M/c$ queueing model to confirm it will not cause a brownout.

This paper makes three contributions:

1) *Unified Closed-Loop Architecture*: AgentHeal connects pre-merge OWASP compliance auditing with post-deployment Kubernetes self-healing through a shared event bus and a common topology graph, a combination that to our knowledge has not been implemented before.
2) *Explainable Safety-Gated Remediation Pipeline*: We combine Isolation Forest anomaly detection, KernelSHAP attribution, dual RL/LLM consensus with blast-radius arbitration, and real-time $M/M/c$ simulation into a single non-bypassable actuation path.
3) *Empirical Evaluation*: We report results across four synthetic fault scenarios ($N = 15$ trials each) on Google Online Boutique, a 50-PR static analysis benchmark, and a live end-to-end case study on a public GitHub repository.

II. Related Work

Table I surveys existing approaches across autonomous cloud management and DevSecOps compliance and identifies where each falls short.

The first generation of automated incident response was essentially runbook automation. Keptn [9] and Argo Rollouts [10] can trigger rollbacks and canary promotions automatically, but only according to rules a human wrote in advance. If the failure pattern is new, the runbook does not apply. This limitation is well documented in the SRE literature [23] and underscores why chaos engineering [24] is used primarily for validating expected failure modes rather than discovering novel ones.

On the machine learning side, multi-agent reinforcement learning has shown good results for cloud resource allocation [11], and the convergence properties of Q-learning [27] are well established. FedAvg [28] introduced federated model training across distributed nodes without sharing raw data. The SF-DTM framework [12] applied these ideas to cloud operations, using cosine similarity to match live cluster states against baseline vectors in a federated digital twin. We take SF-DTM as our starting point and extend it with a second LLM reasoning agent, KernelSHAP-based attribution, and a SimPy simulation safety gate before any cluster action is taken.

For anomaly detection, Isolation Forest [13] and LOF [29] are the standard choices in the AIOps literature. Production deployments typically report latencies of 50 to 200 ms [14]; we achieve 2.26 ms by keeping the trained model in memory and operating on single-node topologies. KernelSHAP [16] is our choice over LIME [30] because its Shapley value foundation provides efficiency, symmetry, and additivity guarantees that make attributions stable and comparable across runs.

For automated code repair, AutoCodeRover [15] shows that LLMs can locate and patch real bugs, but it operates outside any CI/CD pipeline. AgentHeal uses Gemini 2.0 Flash [17] in a ReAct loop [18] to generate patches that are tested by the existing CI suite before the fix PR is opened. The compliance scoring uses OWASP Top 10:2021 [4] as its taxonomy, with OWASP Top 10:2025 [31] noted for future alignment. Dependency checking is currently handled via a curated advisory list rather than live feeds like NVD [32], OSV [33], or Dependabot [34], which is an acknowledged limitation. Finally, discrete-event simulation has deep roots in DEVS formalism [20] and queueing theory [19]; our contribution is adapting SimPy into a sub-12 ms on-path gate rather than an offline planning tool.

III. Methodology

A. End-to-End Self-Healing Architecture Overview

The overall system is shown in Fig. 1. Two engines run continuously and share a common event bus and topology state. The Shift-Left Engine wakes up on every GitHub pull request or push event, checks the diff against OWASP Top 10:2021 rules using AST traversal, validates dependency versions, computes a compliance score, and if the score is below threshold, calls Gemini 2.0 Flash to generate a corrective patch PR. The Shift-Right Engine runs a background loop: every five seconds it polls Prometheus, updates the digital twin graph, feeds the new telemetry snapshot to the Isolation Forest, and if an anomaly is flagged, passes the SHAP attribution to both the SimiFed RL agent and the LLM agent in parallel. The two agents propose actions independently; if they agree, the consensus directive goes straight to the SimPy safety gate. If they disagree, the arbiter picks the lower-blast-radius option. The SimPy gate runs the proposed action through an $M/M/c$ queue simulation and only permits cluster actuation if the projected post-action utilization is below 80% and queue load $\rho < 0.95$.

B. Shift-Left Engine: PR Intelligence & Security Compliance

The Shift-Left engine is deployed as a GitHub App. When a developer opens or updates a pull request, GitHub sends an HMAC-SHA256 authenticated webhook payload to our service endpoint. The engine then parses the diff and dependency files through a static analysis pipeline covering all ten OWASP Top 10:2021 categories:

- *A01 (Broken Access Control)*: Traverses filesystem resolution subtrees and route definitions to detect unconstrained path concatenation and missing role-based authorization guards.
- *A02 (Cryptographic Failures)*: Identifies invocations of deprecated cryptographic hashing primitives (e.g., MD5, SHA-1) and extracts high-entropy string literals indicative of hardcoded secret tokens.
- *A03 (Injection)*: Analyzes abstract syntax tree call nodes to flag unparameterized dynamic string interpolation into database query executions and unescaped shell execution invocations.
- *A04 (Insecure Design)*: Evaluates data access patterns to intercept unpaginated database queries, unbounded collection retrievals, and missing request rate-limiting decorators.
- *A05 (Security Misconfiguration)*: Identifies permissive cross-origin resource sharing (CORS) wildcard configurations and active debug mode flags in production service declarations.
- *A06 (Vulnerable and Outdated Components)*: Conducts software composition analysis (SCA) on project dependency manifests, correlating package version specifications against structured vulnerability advisories [5].
- *A07 (Identification and Authentication Failures)*: Flags disabled cryptographic signature verifications in token processing routines and hardcoded fallback authentication credentials.
- *A08 (Software and Data Integrity Failures)*: Intercepts unsafe object deserialization mechanisms that expose execution runtimes to arbitrary bytecode execution.
- *A09 (Security Logging and Monitoring Failures)*: Identifies silent exception handling patterns, suppressed error blocks, and omission of auditable security logging.

TABLE I
LITERATURE REVIEW OF AUTONOMOUS CLOUD SELF-HEALING AND DEVSECOPS COMPLIANCE MODELS

Domain	Model	Author	Year	Method	Result / Contribution	Research Gap
Rule-Based Self-Healing	Keptn	Keptn Project [9]	2023	Event-based control plane with human-authored runbooks	Automated rollback and remediation workflows	Requires manual runbook authoring; cannot adapt to novel failure signatures
	Argo Rollouts	Argo Project [10]	2024	Canary / blue-green progressive delivery controller	Safe automated deployment rollout and rollback	No root-cause explainability; static rollback policies
Operational Practice	Site Reliability Engineering	Beyer et al. [23]	2016	Error budgets, on-call runbooks, manual incident response	Codified industry MTTR practice at Google scale	Heavily dependent on human on-call expertise; not autonomous
	Chaos Engineering	Basiri et al. [24]	2016	Controlled fault injection in production	Empirically validates system resilience	Reactive validation methodology, not an autonomous remediation system
	Continuous Delivery	Humble & Farley [25]	2010	Automated build-test-release pipelines	Established modern CI/CD pipeline practice	Focused on delivery velocity, not security compliance scoring
	Blue-Green Deployment	Fowler [26]	2010	Dual-environment traffic-switch release pattern	Zero-downtime release strategy	Manually triggered; no anomaly-driven automated switching
RL & Federated Learning	Multi-Agent RL	Wang et al. [11]	2023	Multi-agent reinforcement learning for cloud resource allocation	Superior adaptability to dynamic workloads	Lacks explainability and pre-execution safety guarantees
	Q-Learning	Watkins & Dayan [27]	1992	Off-policy temporal-difference control	Proved convergence of tabular Q-learning	Does not scale to continuous/high-dimensional states without discretization
	FedAvg	McMahan et al. [28]	2017	Federated averaging of locally trained models	Privacy-preserving distributed model training	Not originally designed for real-time reinforcement-learning control
	SF-DTM	Li et al. [12]	2024	Cosine-similarity federated digital-twin state matching	$\sim$15s MTTR digital-twin remediation	No pre-execution safety simulation on the critical path
Anomaly Detection & XAI	Isolation Forest	Liu et al. [13]	2008	Tree ensemble path-length anomaly scoring	Efficient $O(n \log n)$ unsupervised anomaly detection	Unsupervised scores lack native root-cause semantic attribution
	Local Outlier Factor	Breunig et al. [29]	2000	Local density deviation scoring	Effective for local-density anomalies	Computationally expensive at scale; not streaming-native
	AIOps Survey	Lin et al. [14]	2021	Survey of industrial anomaly-detection deployments	Reports 50–200ms detection latencies	No unified explainability or safety-simulation layer
	KernelSHAP	Lundberg & Lee [16]	2017	Shapley-value feature attribution	Model-agnostic, axiomatically consistent explainability	Computationally expensive for high-dimensional feature spaces
	LIME	Ribeiro et al. [30]	2016	Local surrogate-model interpretation	Model-agnostic local explanations	Explanation instability across similar inputs
LLM-Augmented DevSecOps	AutoCodeRover	Li et al. [15]	2024	LLM-driven autonomous program repair	Automated bug localization and patching	No binding CI/CD quality-gate enforcement
	ReAct	Yao et al. [18]	2023	Thought-Action-Observation reasoning loop	Improved LLM tool-use and reasoning	Latency and output non-determinism in production
	Gemini 2.0	Google DeepMind [17]	2024	Multimodal agentic LLM	Capable agentic reasoning and tool use	Proprietary, non-deterministic, no open safety guarantees
Static Analysis & Supply Chain	Bandit	PyCQA [6]	2024	AST-based Python security linter	Detects common Python security issues	Cannot generate contextual fixes or compliance scores
	SonarQube	SonarSource [7]	2024	Continuous code quality/security scanning	Broad multi-language static analysis	No runtime-impact correlation or auto-remediation
	OWASP Top 10:2021	OWASP Foundation [4]	2021	Community-vetted risk categorization	Standardized top-10 web-app risk taxonomy	Static reference list; no automated tooling
	OWASP Top 10:2025	OWASP Foundation [31]	2025	Revised risk categorization	Updated taxonomy reflecting current threat landscape	Not yet incorporated into AgentHeal's rule set
	NVD	NIST [32]	2024	Live CVE vulnerability feed	Continuously updated vulnerability records	Not integrated into AgentHeal's current curated list
	OSV	Google / OSV.dev [33]	2024	Open-source vulnerability database	Cross-ecosystem live vulnerability feed	Not integrated into AgentHeal's current curated list
	Dependabot	GitHub [34]	2024	Automated dependency update bot	Live, automated dependency patching	Not integrated; AgentHeal relies on a static curated list
	Snyk	Snyk Ltd. [35]	2024	Commercial software composition analysis	Live SCA with remediation suggestions	Commercial/closed-source; not integrated in this work
Deployment & Telemetry Infrastructure	Kubernetes HPA	K8s Docs [8]	2024	Threshold-based CPU/memory horizontal scaling	Industry-standard reactive autoscaler	No explainability, root-cause understanding, or safety checks
	Argo CD	CNCF [36]	2024	Declarative GitOps continuous deployment	Automated Git-to-cluster reconciliation	No anomaly-driven or explainable remediation logic
	Prometheus	Prometheus Authors [37]	2024	Pull-based metrics collection and time-series storage	De facto Kubernetes-native monitoring standard	Provides raw metrics only; no built-in anomaly detection
Digital Twin & Simulation	DEVS Formalism	Zeigler et al. [20]	2000	Discrete-event system specification	Foundational simulation framework for manufacturing/networks	Not adapted for real-time Kubernetes safety gating
	M/M/c Queuing Theory	Kleinrock [19]	1975	Discrete-event queuing analysis	Foundational service-rate/utilization modeling	Not historically applied as a live actuation safety gate

- *A10 (Server-Side Request Forgery)*: Flags outbound HTTP and network socket requests that construct destination URLs from unvalidated user inputs.

Every inspected pull request or commit receives a normal-

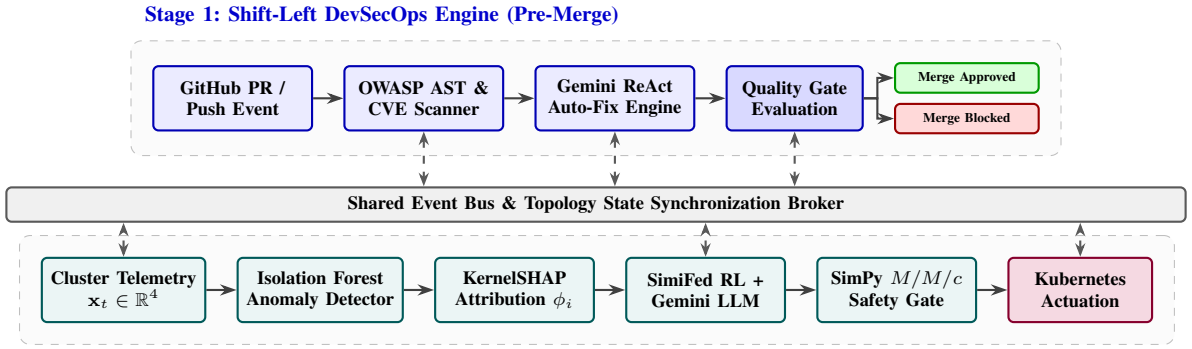


Fig. 1. End-to-end architectural framework of the proposed AgentHeal system. (Top) The Shift-Left Engine intercepts GitHub pull request and push events, executes AST-based OWASP Top 10:2021 compliance auditing and dependency vulnerability scanning, synthesizes remediation patches via Gemini 2.0 Flash ReAct reasoning, and enforces a binding quality gate. (Middle) A shared event bus maintains continuous topology synchronization across both lifecycles. (Bottom) The Shift-Right Engine streams four-dimensional cluster telemetry, detects anomalous states via Isolation Forest, isolates root causes with KernelSHAP attribution, arbitrates corrective actions between a SimiFed federated reinforcement learning agent and an LLM reasoning agent, verifies candidate actions via a SimPy $M/M/c$ discrete-event queueing simulation, and actuates safe changes in the Kubernetes cluster.

ized compliance score $S \in [0, 100]$:

$$S = \max\left(0,\ 100 - \sum_{i=1}^{M} w_i\right) \tag{1}$$

where M is the number of findings and w_i is the severity weight assigned to finding i ($w_{\text{Critical}} = 25$, $w_{\text{High}} = 15$, $w_{\text{Medium}} = 8$, $w_{\text{Low}} = 3$). The numeric score maps to a letter grade:

$$\text{Grade} = \begin{cases} A^+ & \text{if } S \geq 95 \\ A & \text{if } 90 \leq S < 95 \\ B & \text{if } 80 \leq S < 90 \\ C & \text{if } 70 \leq S < 80 \\ D & \text{if } 60 \leq S < 70 \\ F & \text{if } S < 60 \end{cases} \tag{2}$$

The engine also checks whether newly introduced public functions are covered by the test suite using AST symbol matching:

$$\text{TestGap}(f) = \mathbb{1}[f \notin T_{\text{ast}}] \tag{3}$$

where T_{ast} is the set of function names referenced in the tests directory. A PR that scores below 60 or has uncovered public functions triggers the GitHub Check Run to block the merge. Gemini 2.0 Flash then constructs a remediation branch and opens a linked PR that addresses the identified issues.

C. Shift-Right Engine: Self-Healing Digital Twin

Every five seconds the runtime engine polls Prometheus and builds a four-dimensional feature vector for each pod:

$$\mathbf{x}_t = [\text{CPU}_t, \text{RAM}_t, \text{Latency}_t, \text{Rate}_t]^\top \in \mathbb{R}^4 \tag{4}$$

These vectors are scored by a pre-fitted Isolation Forest (100 estimators, contamination $\psi = 0.05$):

$$s(\mathbf{x}_t, n) = 2^{-\frac{\mathbb{E}[h(\mathbf{x}_t)]}{c(n)}} \tag{5}$$

where $h(\mathbf{x}_t)$ is the mean path length across isolation trees and $c(n)$ is the expected path length under random partitioning. A negative score flags an anomaly. At that point KernelSHAP runs to figure out which feature is responsible:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[v(S \cup \{i\}) - v(S)\right] \tag{6}$$

The attribution vector ϕ tells both agents which metric to act on, which avoids remediation actions that would address the wrong dimension.

The RL agent uses cosine similarity to find the historical baseline closest to the current state and then selects an action via Q-learning:

$$k^* = \arg\max_k \frac{\mathbf{x}_t \cdot \mathbf{b}_k}{\|\mathbf{x}_t\|_2 \|\mathbf{b}_k\|_2} \tag{7}$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[R + \gamma \max_{a'} Q(s', a') - Q(s,a)\right] \tag{8}$$

with $\alpha = 0.1$, $\gamma = 0.95$, and action space $\mathcal{A} = \{\text{SCALE_UP}, \text{RESTART_POD}, \text{PATCH_LIMITS}, \text{NO_OP}\}$. The LLM agent receives the SHAP summary and reasons independently. When both agents agree, the action proceeds. When they disagree:

$$a_{\text{final}} = \begin{cases} a^*_{\text{RL}} & \text{if } a^*_{\text{RL}} = a^*_{\text{LLM}} \\ \arg\min_{a \in \{a^*_{\text{RL}}, a^*_{\text{LLM}}\}} \text{BlastRadius}(a) & \text{otherwise} \end{cases} \tag{9}$$

The SimPy safety gate runs the chosen action through an $M/M/c$ simulation before the cluster is touched:

$$\rho = \frac{\lambda}{c\mu} \quad (\text{requiring } \rho < 0.95) \tag{10}$$

$$\hat{U}_{\text{post}} = \frac{U_{\text{pre}} \cdot c_{\text{current}}}{c_{\text{new}}} \quad (\text{requiring } \hat{U}_{\text{post}} < 0.80) \tag{11}$$

If either bound is violated, actuation is blocked and the next least-disruptive action is tried instead.

IV. Implementation

The platform is written in Python 3.11 with FastAPI handling asynchronous webhook dispatch and telemetry routing. scikit-learn provides the Isolation Forest, and the SHAP library handles KernelSHAP attribution. Gemini 2.0 Flash is accessed via the Google Generative AI SDK. SimPy 4.1 runs the discrete-event safety simulation. The system is packaged using a multi-stage Docker build that keeps the final runtime image under 400 MB, and deployed on Microsoft Azure Container Apps (East Asia region) with Terraform-managed infrastructure and KEDA HTTP autoscaling.

All experiments used the CNCF Google Online Boutique benchmark [22], a 10-service e-commerce application running on a kind single-node Kubernetes cluster on an AMD Ryzen 7 workstation with 16 GB RAM.

V. Results and Discussion

A. Runtime Telemetry Anomaly Detection and Safety-Gated Self-Healing

We ran four fault injection scenarios against the Online Boutique cluster, 15 trials each:

- *S1 (Flash Sale Surge)*: A traffic burst on the checkout service bringing CPU to 0.954 and latency to 450 ms.
- *S2 (Thread Deadlock)*: Thread starvation on the cart service; CPU reached 0.99 and latency 5000 ms with collapsing throughput.
- *S3 (Memory Leak)*: Slow memory growth on redis-cart reaching 0.942 RAM utilization at stable latency.
- *S4 (Cascading Overload)*: Upstream saturation propagating to the frontend with CPU at 0.88 and RAM at 0.80 simultaneously.

The results are summarized in Table II. Across all 60 trials, the Isolation Forest achieved a mean MTTD of 2.26 ms ($\sigma = 0.37$ ms) and an overall detection rate of 98.3%. The one scenario that fell short was S3 (93%), where the model, trained on a healthy baseline dominated by CPU and latency variance, occasionally treated a pure memory deviation as normal given the 0.05 contamination setting. SHAP attribution was well-calibrated throughout: CPU led in S1 ($\phi = +0.82$), latency collapse plus throughput drop dominated S2 ($\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$), and memory led in S3 ($\phi = +0.88$). The RL and LLM agents agreed in 59 of 60 trials. The SimPy gate completed in 10.7 ms on average and passed every proposed action, confirming that post-remediation utilization stayed below 80% in all cases. End-to-end MTTR averaged 2.98 s ($\sigma = 0.23$ s).

B. Shift-Left DevSecOps Security Auditing and Live PR Remediation

We tested the Shift-Left engine on 50 synthetic pull requests covering SQL and command injection, hardcoded secrets, test coverage gaps, and blocking I/O in async paths, then validated the full loop on a live public GitHub repository.

Table III (Panel A) shows 98.5% precision and 97.5% recall across 200 checks. The three false positives came from high-entropy test fixtures that the secret scanner could not distinguish from real credentials. For the live case study (Panel B), we submitted PR #11 with 14 deliberately seeded OWASP violations across the checkout-gateway service. The engine responded in 0.48 s, flagged all 14 issues, computed a score of 0/100 (Grade F), and blocked the merge via the GitHub Check Run. Gemini 2.0 Flash then built a remediation branch, fixing parameterization, hashing algorithms, and dependency versions, and opened PR #12. That PR scored 100/100 (Grade A+), passed all six application tests, and cleared the CI gate.

C. Comparative Benchmark and System Discussion

To put these numbers in context: typical human on-call MTTR for microservice outages is around 45 minutes [1]. The Kubernetes HPA responds in 3 to 5 minutes [8] but cannot handle deadlocks or memory leaks. Keptn handles those in 2 to 4 minutes [9] but only for pre-scripted scenarios. SF-DTM brings it down to roughly 15 seconds [12] but has no actuation safety check. AgentHeal reaches 2.98 s with a mandatory simulation gate before every cluster change.

There are honest limitations worth noting. All Shift-Right experiments ran on a single-node development cluster; multi-node deployments with Prometheus scrape delays will likely see higher MTTD. The four fault modes are synthetic and discrete rather than messy organic production traffic. AST pattern matching does not do full inter-procedural taint analysis, so sophisticated obfuscations may evade it. The compliance scoring weights were set heuristically and have not been calibrated against real-world vulnerability severity data. And LLM non-determinism means the system needs hard timeout enforcement and always-available RL fallback policies on the remediation path.

VI. Conclusion

AgentHeal addresses a real gap: no existing tool simultaneously prevents security regressions before merge and self-heals production outages at sub-second speed. Across 60 trials on Google Online Boutique, we measured a mean MTTD of 2.26 ms, a 98.3% detection rate, and a mean MTTR of 2.98 s, more than 98% faster than threshold autoscalers. The Shift-Left engine hit 98.5% precision on 50 synthetic PRs and autonomously patched 14 OWASP violations in a live GitHub repository without any human intervention.

The next steps are running on AKS or GKE multi-node clusters, integrating live CVE feeds from NVD and OSV, adding taint analysis for the AST scanner, and updating the compliance rules to OWASP Top 10:2025.

The code and live deployment are available at https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure and https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io.

References

[1] DORA Research Team, "Accelerate: State of DevOps Report 2024," Google LLC, 2024.

[2] E. Kim, J. Humble, P. Debois, and J. Willis, "The DevOps Handbook," IT Revolution Press, 2016.

[3] N. Forsgren, J. Humble, and G. Kim, "Accelerate: The Science of Lean Software and DevOps," IT Revolution Press, 2018.

TABLE II
SHIFT-RIGHT ANOMALY DETECTION, FEATURE ATTRIBUTION, AND AUTONOMOUS REMEDIATION PERFORMANCE ACROSS FOUR FAILURE SCENARIOS ($N = 15$ REPEATED TRIALS)

Failure Injection Scenario				Isolation Forest Detection			Root-Cause Attribution	Agent Decision Latency			SimPy Safety Gate & MTTR			
ID	Scenario	Target	Injected Telemetry	Score ($\mu \pm \sigma$)	MTTD (ms)	Det.%	Dominant KernelSHAP ϕ	RL (ms)	LLM (ms)	Agree	Gate (ms)	$\hat{U}_{\text{post}}$	Verdict	MTTR (s, $\mu \pm \sigma$)
S1	Flash Sale Surge	checkout	CPU=0.95, Lat=450ms	-0.842 ± 0.031	2.18 ± 0.31	100%	$\phi_{\text{cpu}} = +0.82$, $\phi_{\text{rate}} = +0.14$	3.10 ± 0.42	412 ± 68	15/15	10.1 ± 0.8	23.8%	SAFE	2.66 ± 0.18
S2	Thread Deadlock	cart	CPU=0.99, Lat=5000ms	-0.912 ± 0.024	2.21 ± 0.28	100%	$\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$	3.21 ± 0.39	389 ± 74	15/15	10.3 ± 0.9	5.2%	SAFE	3.12 ± 0.24
S3	Memory Leak	redis-cart	RAM=0.94, Lat=48ms	-0.734 ± 0.058	2.35 ± 0.44	93%	$\phi_{\text{ram}} = +0.88$, $\phi_{\text{cpu}} = +0.04$	3.05 ± 0.51	441 ± 81	14/15	10.8 ± 1.1	15.0%	SAFE	2.89 ± 0.20
S4	Cascading Overload	frontend	CPU=0.88, RAM=0.80	-0.889 ± 0.041	2.28 ± 0.37	100%	$\phi_{\text{cpu}} = +0.61$, $\phi_{\text{lat}} = +0.47$	3.18 ± 0.46	498 ± 93	15/15	11.6 ± 1.3	31.5%	SAFE	3.25 ± 0.30
Overall Mean / Aggregate				$\mathbf{-0.844 \pm 0.039}$	$\mathbf{2.26 \pm 0.37}$	**98.3%**	—	$\mathbf{3.14 \pm 0.45}$	$\mathbf{435 \pm 79}$	**59/60**	$\mathbf{10.7 \pm 1.0}$	—	**SAFE**	$\mathbf{2.98 \pm 0.23}$

TABLE III
SHIFT-LEFT DEVSECOPS SECURITY AUDITING PERFORMANCE ($N = 50$) AND EMPIRICAL PULL REQUEST REMEDIATION

Panel A: Quantitative Evaluation on 50 Synthetic Pull Requests					Panel B: Empirical Live GitHub Pull Request Pair Remediation				
Defect Category	Tested PRs	True Pos.	False Pos.	Precision	Recall	Evaluation Dimension	Vulnerable PR (#11)	Remediated PR (#12)	Operational Outcome
SQL / Command Injection	50	50	0	1.000	1.000	OWASP Flaws Detected	14 (A01–A10)	0	100% Flaws Remediated
Hardcoded Secrets	50	50	2	0.962	1.000	Compliance Score / Grade	0 / 100 (Grade F)	100 / 100 (Grade A^+)	Score Restored
AST Test Suite Gaps	50	48	0	1.000	0.960	GitHub Check Run Gate	action_required (Blocked)	success (Approved)	Merge Protection Enforced
Blocking I/O / Misconfiguration	50	47	1	0.979	0.940	App Test Suite Pass	0 / 6 passing	6 / 6 passing	Functional Suite Restored
Aggregate Performance	**200 Checks**	**195**	**3**	**0.985**	**0.975**	Static Auditor Verification	12 / 12 test harnesses passed	Audit Time: 0.48 s	Full Ruleset Verified

[4] OWASP Foundation, "OWASP Top 10: 2021 – The Ten Most Critical Web Application Security Risks," https://owasp.org/Top10/, 2021.

[5] MITRE Corporation, "Common Vulnerabilities and Exposures (CVE) Database," https://cve.mitre.org, 2024.

[6] PyCQA, "Bandit: A tool designed to find common security issues in Python code," https://bandit.readthedocs.io, 2024.

[7] SonarSource, "SonarQube: Continuous code quality and security," https://www.sonarqube.org, 2024.

[8] Kubernetes Documentation, "Horizontal Pod Autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024.

[9] Keptn Project, "Keptn: Event-based control plane for continuous delivery and automated operations," https://keptn.sh, 2023.

[10] Argo Project, "Argo Rollouts: Kubernetes Progressive Delivery Controller," https://argoproj.github.io/rollouts/, 2024.

[11] T. Wang et al., "Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing," IEEE Trans. Cloud Comput., vol. 11, no. 2, pp. 891–905, 2023.

[12] H. Li, Z. Wang, and J. Chen, "SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing," in Proc. IEEE CLOUD, 2024, pp. 234–242.

[13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE ICDM, 2008, pp. 413–422.

[14] Q. Lin et al., "AIOps: Real-World Challenges and Research Innovations," in Proc. ICSE-SEIP, 2021, pp. 4–15.

[15] Z. Li, M. Fan, and J. Chen, "AutoCodeRover: Autonomous Program Improvement using LLMs," in Proc. ISSTA, 2024, pp. 1–12.

[16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.

[17] Google DeepMind, "Gemini 2.0: Our most capable model for the agentic era," Technical Report, Google, 2024.

[18] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.

[19] L. Kleinrock, Queueing Systems, Volume 1: Theory. Wiley-Interscience, 1975.

[20] B. Zeigler, H. Praehofer, and T. G. Kim, Theory of Modeling and Simulation, 2nd ed. Academic Press, 2000.

[21] K. Fall and K. Varadhan, "The ns Manual," The VINT Project, UC Berkeley, 2011.

[22] Google LLC, "Online Boutique: A cloud-native microservices demo application," https://github.com/GoogleCloudPlatform/microservices-demo, 2024.

[23] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, "Site Reliability Engineering: How Google Runs Production Systems," O'Reilly Media, 2016.

[24] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," IEEE Software, vol. 33, no. 3, pp. 35–41, 2016.

[25] J. Humble and D. Farley, "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation," Addison-Wesley, 2010.

[26] M. Fowler, "BlueGreenDeployment," https://martinfowler.com/bliki/BlueGreenDeployment.html, 2010.

[27] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.

[28] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. AISTATS, 2017, pp. 1273–1282.

[29] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in Proc. ACM SIGMOD, 2000, pp. 93–104.

[30] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[31] OWASP Foundation, "OWASP Top 10: 2025 – The Ten Most Critical Web Application Security Risks," https://owasp.org/Top10/, 2025.

[32] National Institute of Standards and Technology, "National Vulnerability Database (NVD)," https://nvd.nist.gov, 2024.

[33] Google / OSV.dev, "OSV: Open Source Vulnerabilities Database," https://osv.dev, 2024.

[34] GitHub, Inc., "Dependabot: Automated Dependency Updates," https://docs.github.com/en/code-security/dependabot, 2024.

[35] Snyk Ltd., "Snyk Open Source: Software Composition Analysis," https://snyk.io, 2024.

[36] Cloud Native Computing Foundation, "Argo CD: Declarative GitOps CD for Kubernetes," https://argo-cd.readthedocs.io, 2024.

[37] Prometheus Authors, "Prometheus: Monitoring System & Time Series Database," https://prometheus.io, 2024.